

SN360/SN361 Introduction à la conception des circuits numériques

Module « Systèmes matériels et logiciels »

Crédit SN360 : 4/4 / SN361 : 2/6

Vincent Berouille

Bureau : D202

vincent.berouille@esisar.grenoble-inp.fr

Plan

I Introduction

II Représentation des nombres en binaire

III Circuits combinatoires

IV Circuits séquentiels

V Machines à états finis

VI Compléments sur les HDL

VII Circuits reconfigurables (optionnel)

V Machine à états finis

Définitions

- Automates ou Machine à états finis (Finite State Machine = FSM) très utilisés dans de nombreux domaines (informatique, automatique, etc)
- Représentation graphique simple d'un système discret à entrées/sorties
- On s'intéresse ici à leurs réalisations à l'aide de **circuits numériques séquentiels**
- 2 principaux types de FSM :
 - **Machine de Moore**
 - **Machine de Mealy**

V Machine à états finis

Représentation : diagramme de transitions

- On utilise un **graphe d'états** ou **diagramme de transitions** :
 - État : 1 cercle + 1 nom (le nom de l'état)
 - État initial : 1 double cercle + 1 flèche "venant de nulle part" pour le rst
 - Transition : 1 flèche + 1 condition
la transition n'est franchie (changement d'état) que lors du front actif de l'horloge et lorsque la condition de la transition est vraie
 - Sortie :
 - soit associée à l'état (Moore)
 - soit associée à une transition (e/s) (Mealy)

V Machine à états finis

Représentation : diagramme de transitions

Ici I_1 différent de I_2

- Exemples :

Par défaut : $S = 0$

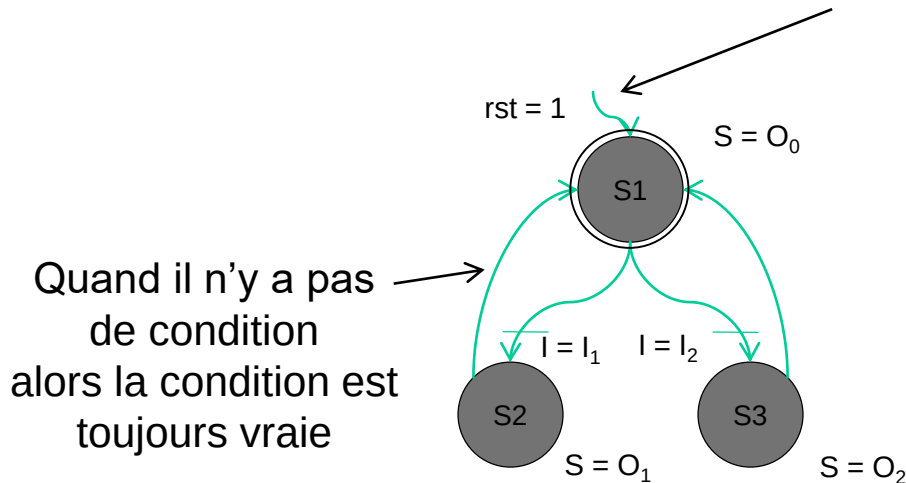


Diagramme de transitions d'une **machine de Moore** :
1 état = 1 sortie

Par défaut : $S = 0$

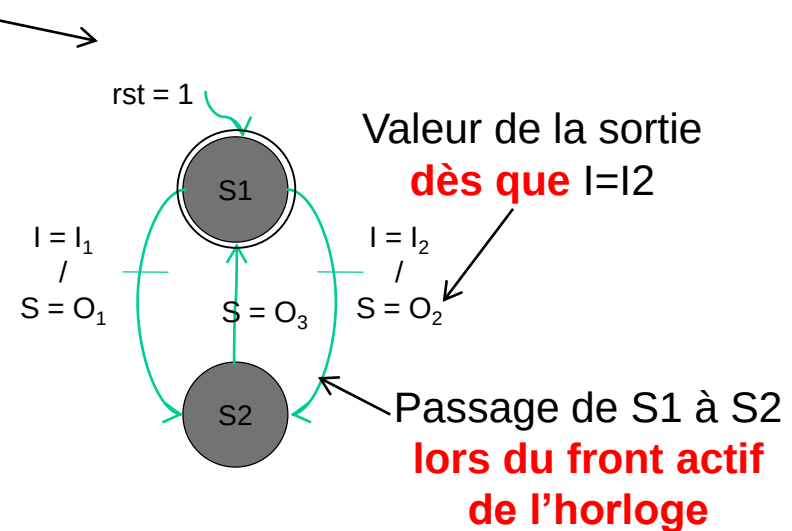


Diagramme de transitions d'une **machine de Mealy** :
1 transition = 1 sortie

Nombre min de FF = $n = \lceil \log_2(\text{nombre d'états}) \rceil$

V Machines à états finis

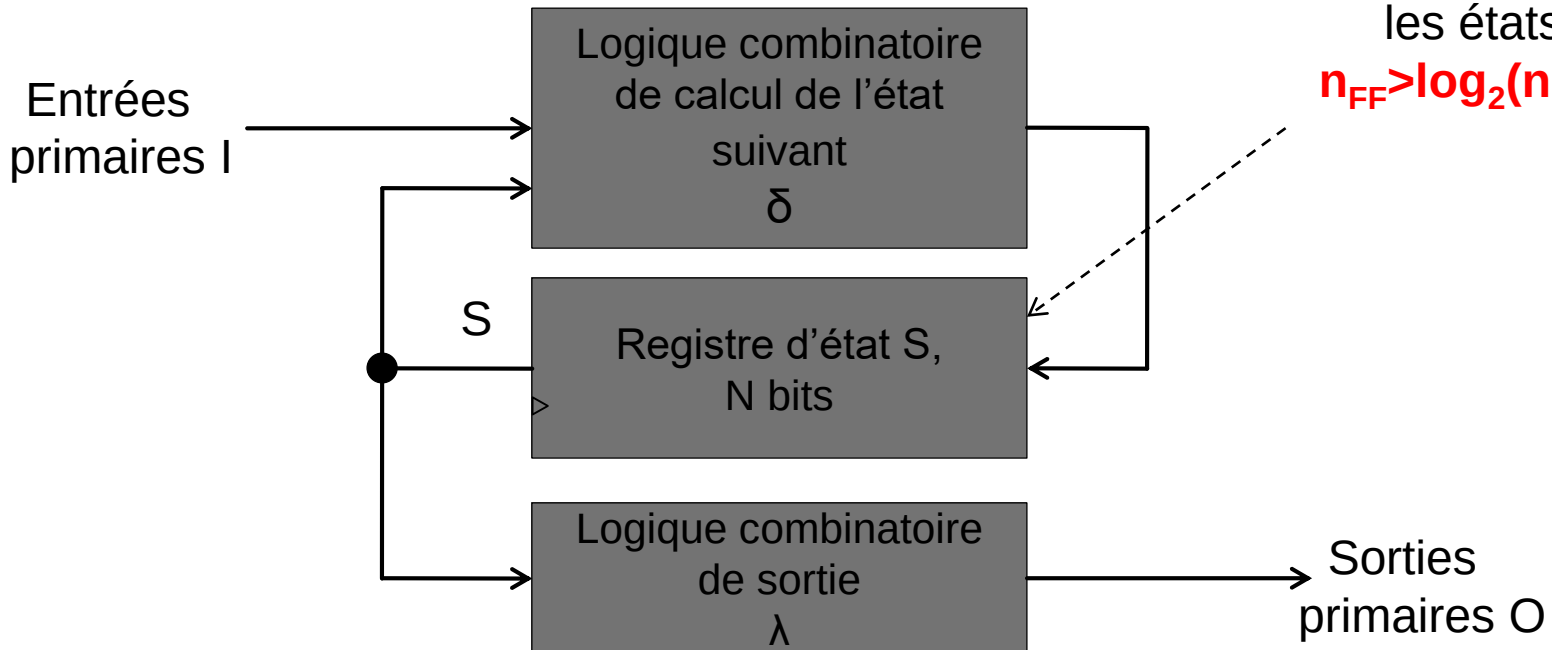
Architecture Machine de Moore

- Sorties calculées uniquement avec l'état courant :

□ $\lambda : S \rightarrow O$

Le nombre de Flip-Flop doit être suffisant pour permettre d'identifier tous les états

$$n_{FF} > \log_2(n_{Etat})$$

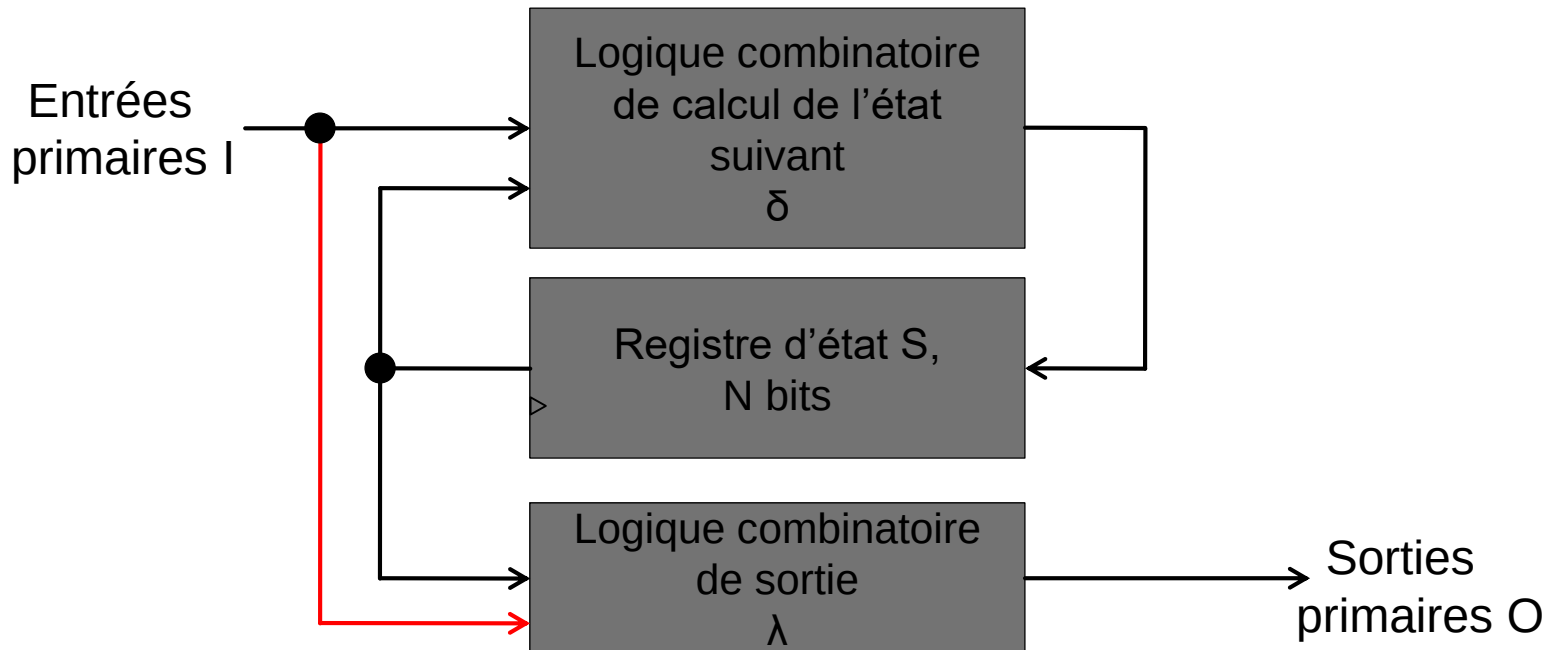


Dans une machine de Moore la sortie dépend seulement de l'état :
O ne peut changer que suite à un front actif de l'horloge

V Machines à états finis

Architecture Mealy

- Sorties calculées avec l'état courant et les entrées primaires I :
 - $\lambda : S \times I \rightarrow O$

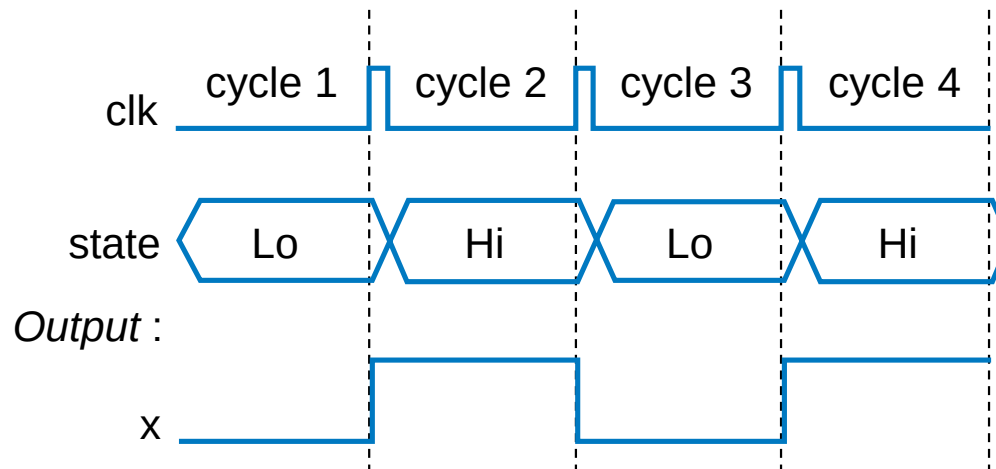
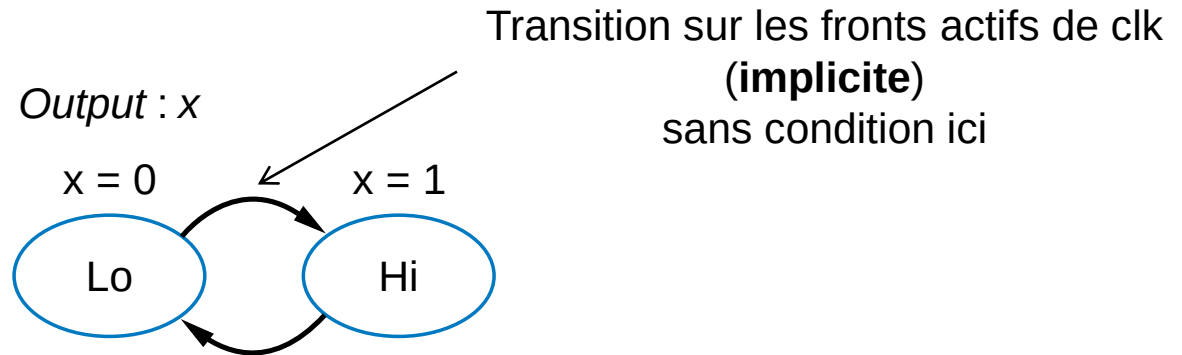


**Dans une machine de Mealy la sortie dépend de l'état et des entrées :
O peut changer sans front actif de l'horloge mais sur un événement d'une entrée**

V Machine à états finis

Exemple

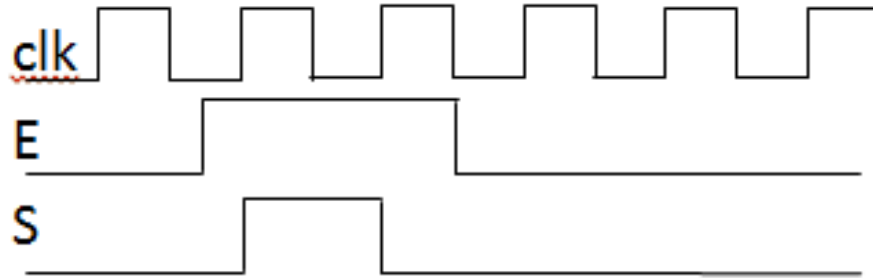
Diagramme de transitions
d'une machine de Moore :



V Machines à états finis

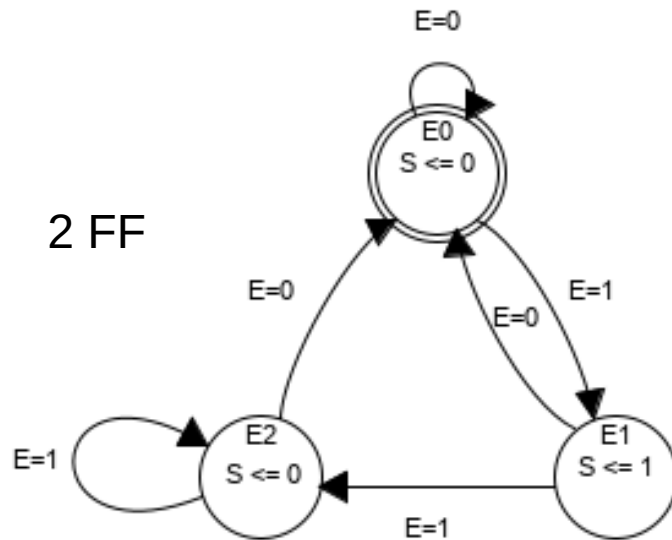
Exercices

- Détecteur de front montant : RisingEdgeDetector
- Avec reset synchrone

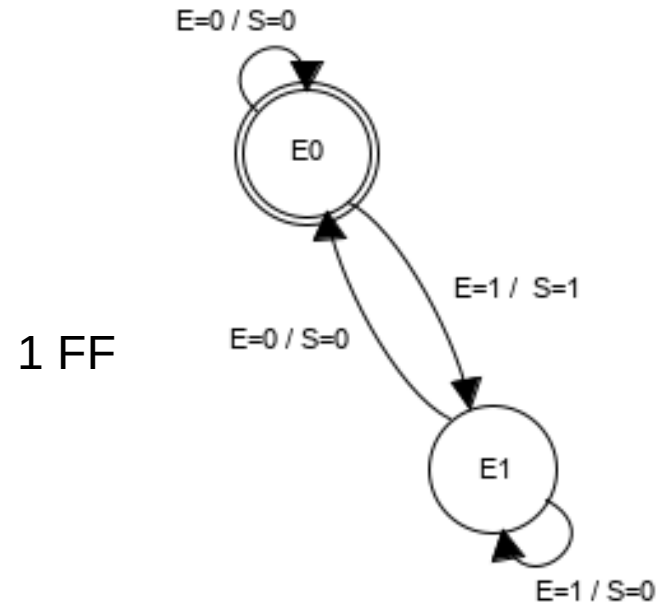
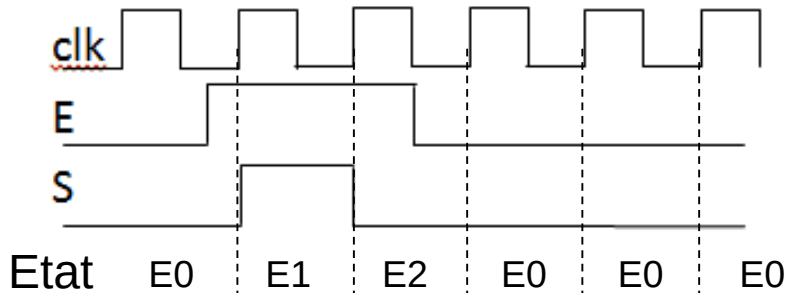


V Machines à états finis

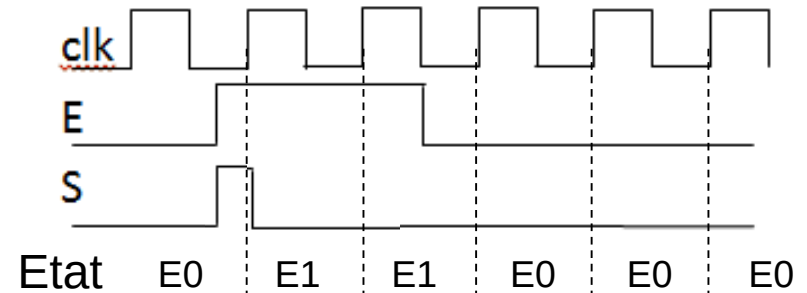
Exercices : détecteur de front montant



Machine de Moore



Machine de Mealy



La machine de Mealy est toujours au moins autant ou plus rapide et au moins autant ou plus petite que celle de Moore

V Machines à états finis

Code VHDL – Machine de Moore (1)

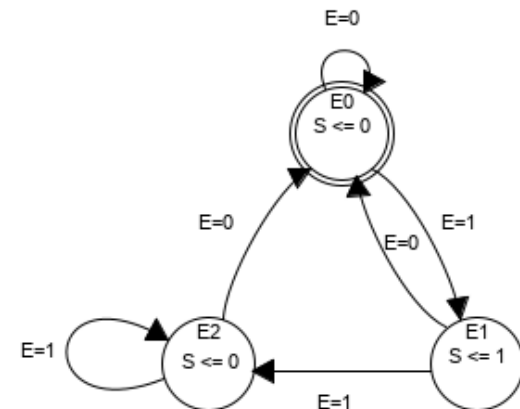
VHDL - Moore

```
entity RisingEdgeDetector is  
port (clk, rst, e : in bit;  
       s : out bit);  
end RisingEdgeDetector;  
architecture RTL_Moore of  
    RisingEdgeDetector is  
    -- déclaration d'un type pour  
    les signaux d'état  
    type state_type is (E0, E1, E2);  
    signal state, next_state : state_type;  
    begin
```

-- processus synchrone pour décrire
le registre

```
process (clk)  
    begin  
        if rising_edge(clk) then  
            if reset = '1' then  
                state <= E0;  
            else  
                state <= next_state;  
            end if;  
        end if;  
    end process;
```

-- voir suite



V Machines à états finis

Code VHDL – Machine de Moore (2)

VHDL Moore - suite

--processus asynchrone ou
 combinatoire

process (state, e)

begin

 next_state <= state;

 s <= '0';

case state is

when E0 =>

if e = '1' **then**

 next_state <= E1;

end if;

when E1 => s <= '1';

if e = '1' **then**

 next_state <= E2;

else

 next_state <= E0;

end if;

when E2 =>

if e = '0' **then**

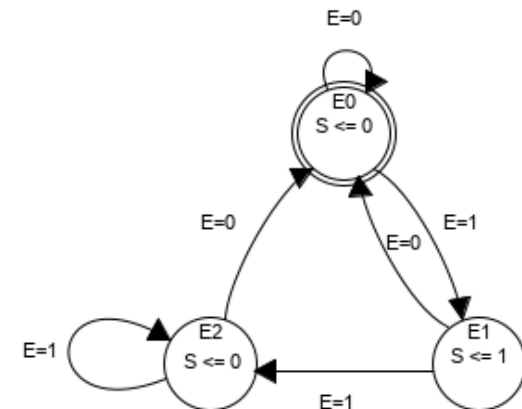
 next_state <= E0;

end if;

end case;

end process;

end RTL_Moore;



V Machines à états finis

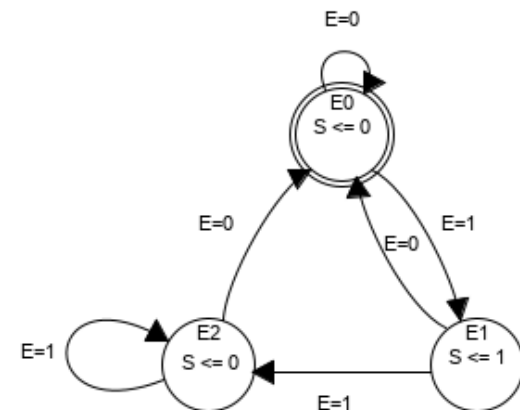
Code Verilog – Machine de Moore (1)

Verilog - Moore

```
module RisingEdgeDetector (  
    input wire clk,    // Horloge  
    input wire rst,    // Reset synchrone  
    input wire e,      // Entrée  
    output reg s       // Sortie  
);  
// Déclaration des signaux d'état  
reg [1:0] state_current, next_state;  
parameter    E0=2'b00,    E1=2'b01,  
             E2=2'b10;
```

// Processus séquentiel pour la mise à jour de l'état

```
always @(posedge clk) begin  
    if (rst == 1'b1) begin  
        current_state <= E0;  
    end else begin  
        current_state <= next_state;  
    end  
end
```



V Machines à états finis

Code Verilog – Machine de Moore (2)

Verilog Moore - suite

// Processus combinatoire pour la logique de la machine d'état

```
always @(*) begin
```

```
    // Valeurs par défaut
```

```
    next_state = current_state;
```

```
    s = 1'b0;
```

```
    case (current_state)
```

```
        E0: begin
```

```
            if (e == 1'b1) begin
```

```
                next_state = E1;
```

```
            end
```

```
        end
```

```
        E1: begin
```

```
            s = 1'b1;
```

```
            if (e == 1'b1) begin
```

```
                next_state = E2;
```

```
            end else begin
```

```
                next_state = E0;
```

```
            end
```

```
        end
```

```
        E2: begin
```

```
            if (e == 1'b0) begin
```

```
                next_state = E0;
```

```
            end
```

```
        end
```

```
        default: begin
```

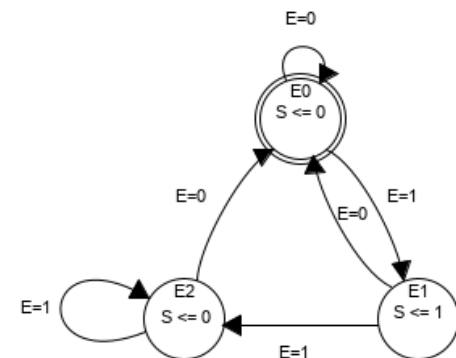
```
            next_state = E0;
```

```
        end
```

```
    endcase
```

```
end
```

```
endmodule
```



V Machines à états finis

Code VHDL – Machine de Mealy (1)

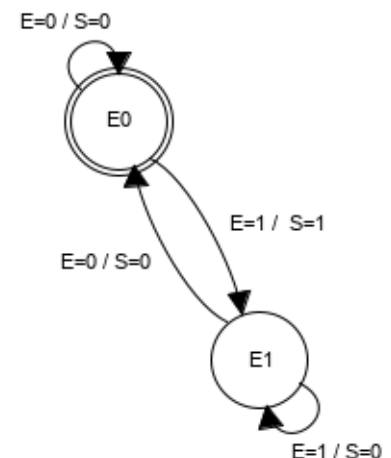
VHDL - Mealy

```
entity RisingEdgeDetector is
  port (clk, rst, e : in bit;
        s : out bit);
end RisingEdgeDetector;
architecture RTL_Mealy of
  RisingEdgeDetector is
    -- déclaration d'un type pour
    les signaux d'état
    type state_type is (E0, E1, E2);
    signal state, next_state : state_type;
  begin
```

-- processus synchrone pour décrire
le registre

```
process (clk)
begin
  if rising_edge(clk) then
    if reset = '1' then
      state <= E0;
    else
      state <= next_state;
    end if;
  end if;
end process;
```

-- voir suite



V Machines à états finis

Code VHDL – Machine de Mealy (2)

VHDL Moore - suite

--processus asynchrone ou
 combinatoire

process (state, e)

begin

next_state <= state;

s <= '0';

case state is

when E0 =>

if e = '1' **then**

 s <= '1';

 next_state <= E1;

end if;

when E1 => s <= '0';

if e = '0' **then**

 next_state <= E0;

end if;

~~**when** E2 =>~~

~~**if** e = '0' **then**~~

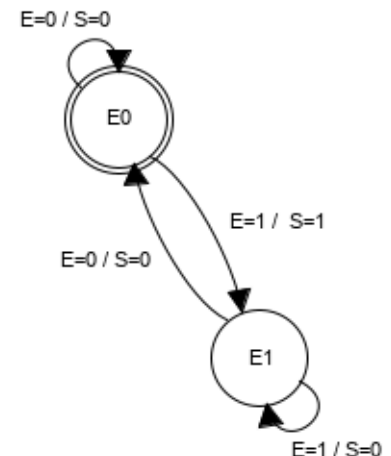
~~next_state <= E0;~~

~~**end if;**~~

end case;

end process;

end RTL_Mealy;



V Machines à états finis

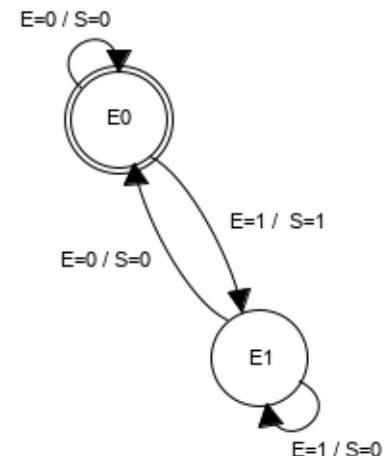
Code Verilog – Machine de Mealy (1)

Verilog - Mealy

```
module RisingEdgeDetector (  
    input wire clk,    // Horloge  
    input wire rst,    // Reset synchrone  
    input wire e,      // Entrée  
    output reg s        // Sortie  
);  
// Déclaration des signaux d'état  
reg [1:0] state_current, next_state;  
parameter E0=1'b0, E1=1'b1, E2=2'b10;
```

// Processus séquentiel pour la mise à jour de l'état

```
always @(posedge clk) begin  
    if (rst == 1'b1) begin  
        current_state <= E0;  
    end else begin  
        current_state <= next_state;  
    end  
end
```



V Machines à états finis

Code Verilog – Machine de Mealy (2)

Verilog Moore - suite

// Processus combinatoire pour la logique de la machine d'état

```
always @(*) begin
```

```
    // Valeurs par défaut
```

```
    next_state = current_state;
```

```
    s = 1'b0;
```

```
case (current_state)
```

```
    E0: begin
```

```
        if (e == 1'b1) begin
```

```
            s = 1'b1;
```

```
            next_state = E1;
```

```
        end
```

```
    end
```

```
E1: begin
```

```
    s = 1'b0;
```

```
    if (e == 1'b0) begin
```

```
        next_state = E2;
```

```
    end else begin
```

```
        next_state = E0;
```

```
    end
```

```
end
```

```
E2: begin
```

```
    if (e == 1'b0) begin
```

```
        next_state = E0;
```

```
    end
```

```
end
```

```
default: begin
```

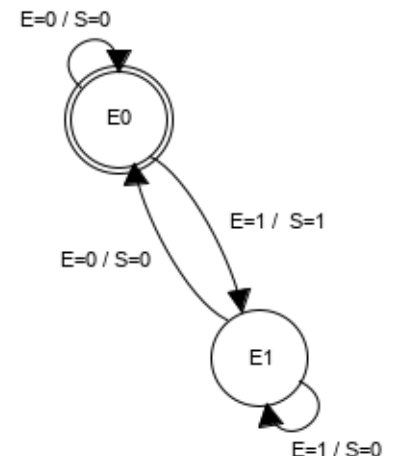
```
    next_state = E0;
```

```
end
```

```
endcase
```

```
end
```

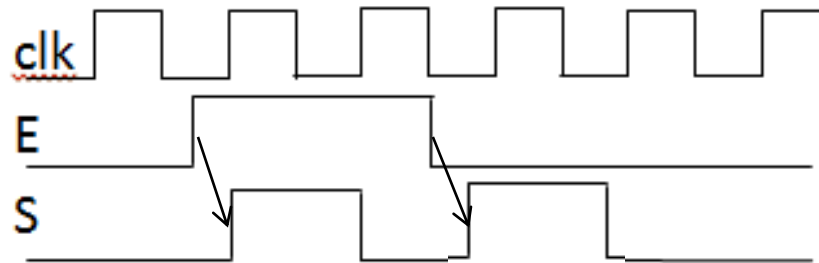
```
endmodule
```



V Machines à états finis

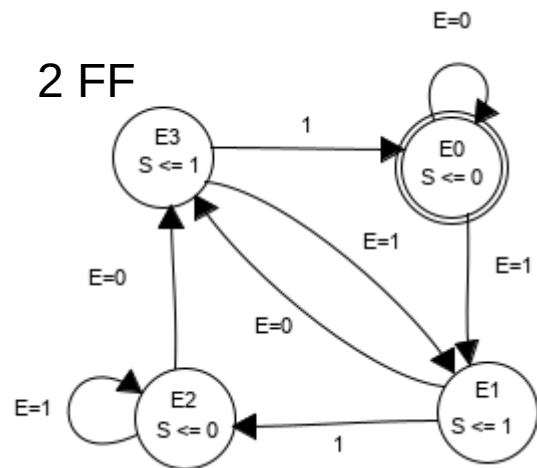
Exercices

- Détecteur de front montant et descendant



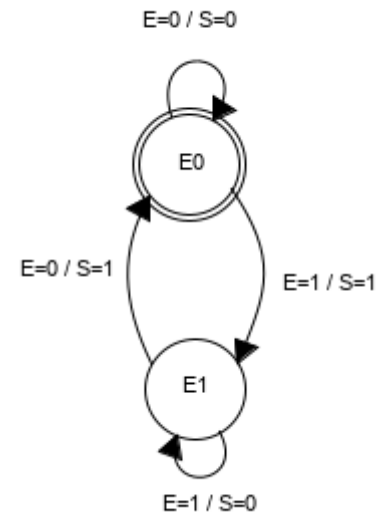
V Machines à états finis

Exercices : détecteur de front montant et descendant



Machine de Moore

1 FF



Machine de Mealy

Vérifiez avec un chronogramme le fonctionnement de ces deux machines

Exercices

- Dessinez puis vérifiez avec des chronogrammes les diagrammes de transitions de Moore et de Mealy des circuits suivants :
 - Détecteur de la séquence d'entrée 101 sans recouvrement, puis avec recouvrement
 - Machines correspondant à l'exemple utilisé pour décrire les circuits séquentiels (avec 2 entrées call & cancel)

Faire le QCM : QCM5 SN360/SN361 Machines à
états finis